

WindBorne Systems, Firmware Intern

Min-En Wu (Brian Wu) minenbrian@gmail.com UCLA Electrical Engineering, Year 3

1. A thing I am proud of

I built an SSVEP brain computer interface that closes the loop from scalp EEG to a physical actuator. OpenBCI streams the EEG. A Python decoder estimates which flicker frequency the user is attending to. An Arduino receives a compact command and drives a bionic hand. The repository is [bw493/ssvep-bci-openbci](https://github.com/bw493/ssvep-bci-openbci).

The unusual part is not the classifier. It is keeping a soft real time path alive across three clocks that do not share a notion of time: the OpenBCI sample clock, the host decode loop, and the Arduino servo loop. A dropped USB chunk or a late classification used to freeze the hand in the last pose. I added sequence numbers, a stale command timeout on the Arduino, and a safe open pose when the link goes quiet. That is the difference between a demo that works on a bench video and a device a person can wear without watching the serial monitor.

This was independent research, not a class project. It sits next to lab work on ESP32 camera pipelines for microfluidic experiments, where the same constraint shows up: limited power, a slow sensor, and a controller that has to fail into a known state.

2. Setup

I write firmware in VS Code with PlatformIO, and host side Python in the same editor. The board is usually an ESP32 or an Arduino class AVR, with a cheap logic analyzer and a USB serial log on the second monitor. I like PlatformIO because the pin map, library lock, and upload command live in the repo, so a lab machine and my laptop build the same binary.

Two years ago I used the Arduino IDE and pasted sketches. That hid include paths and made it hard to diff a fix. PlatformIO plus git is slower on the first day and faster every day after. For a one hour hardware bring up I still drop into the Arduino IDE or a bare esptool flash, because opening a project is not the point.

3. A board that resets every few hours

First I stop changing code. I want one failing unit, a known firmware hash, and a log that survives the reset. I note whether the reset is a brownout, a watchdog, an external pin, or a software restart, from the reset reason register if the chip has one. I also write the reason and the uptime to a retained register or a short flash log so the next boot tells me what the last boot thought happened.

The first tests are electrical, not clever. I scope the 3.3 V rail at the chip, not at the regulator, with a short ground spring, and I look for a dip when the radio, the pump, or the camera turns on. I check the reset pin for a slow rise or a cable that couples in noise. I check that the watchdog is not being starved by a blocking driver. I try a second board. If only one board fails, I suspect a solder joint, a marginal crystal, or a connector. If every board fails on the same duty cycle, I suspect firmware or a load the schematic did not budget.

I change strategy when the reset reason is stable and the rail looks clean for a full event. Then I bisect: disable the radio, then the sensor, then the logging, and see which subsystem moves the interval. A few hour bug is often a counter wrap, a heap leak, or a sensor that wedges the bus until the watchdog fires.

On the microfluidic camera pipeline an ESP32 reset during long runs. The camera and the radio peaked together and the rail dipped just enough to trip the brownout detector. The first fix was staggering the radio window away from the frame capture. The lasting fix was a bulk capacitor at the module and a current budget written next to the pin map, so the next person would not schedule both peaks in the same millisecond.

4. What firmware engineers get wrong

The advice I disagree with is that every driver must be fully abstracted before it is allowed on the board. On a balloon, or on a one off lab rig, the first version should touch the register. A pretty HAL that hides chip errata makes the second bug harder, because the failure is no longer next to the bus transaction that caused it. I would rather have a short, ugly driver with the datasheet page number in the comment, and extract the interface after two chips have survived it.

I also disagree with treating `delay()` as a moral failure. A blocking wait is the right tool while you are proving that a sensor answers. It is the wrong tool once that path shares a core with a control loop or a radio. The mistake is leaving the blocking wait in after the experiment worked. The other common miss is logging only the happy path. In the field the useful byte is the reset reason, the last sensor status, and how full the queue was, not another print of the current altitude.

5. A sketchy fix I am still a little proud of

During a live SSVEP demo the classifier sometimes stalled on a bad USB chunk. The Arduino had already been told to hold a grip, and a stalled host meant the hand stayed closed. I did not have time to redesign the packet layer. I put a 400 ms stale command timeout in the Arduino loop and forced the servo to the open pose if no fresh byte arrived. It is sketchy because the timeout is a magic number, and a slow but healthy host could be told to let go. It kept the demo safe. I later replaced the magic number with a sequence counter and an explicit heartbeat, and I kept the open pose as the fail state. I would ship the fail state again. I would not ship the magic number.

Portfolio: <https://github.com/bw493/ssvep-bci-openbci> and <https://github.com/bw493>. Role applied for: Firmware Intern. Available for in person work in Redwood City, including a summer or quarter internship that can convert.